Red Hat OpenShift AI Self-Managed 3.0

Managing and monitoring models

Manage and monitor models in Red Hat OpenShift AI Self-Managed

Last Updated: 2025-12-16

Red Hat OpenShift AI Self-Managed 3.0 Managing and monitoring models

Manage and monitor models in Red Hat OpenShift AI Self-Managed

Legal Notice

Abstract

As a cluster administrator, you can manage and monitor models in Red Hat OpenShift AI Self-Managed.

Table of Contents

CHAPTER 1. MANAGING MODEL-SERVING RUNTIMES

As a cluster administrator, you can create a custom model-serving runtime and edit the inference service for a model deployed in OpenShift AI.

1.1. ADDING A CUSTOM MODEL-SERVING RUNTIME

A model-serving runtime adds support for a specified set of model frameworks and the model formats supported by those frameworks. You can use the preinstalled runtimes that are included with OpenShift AI. You can also add your own custom runtimes if the default runtimes do not meet your needs.

As an administrator, you can use the OpenShift AI interface to add and enable a custom model-serving runtime. You can then choose the custom runtime when you deploy a model on the model serving platform.

NOTE

Red Hat does not provide support for custom runtimes. You are responsible for ensuring that you are licensed to use any custom runtimes that you add, and for correctly configuring and maintaining them.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have built your custom runtime and added the image to a container image repository such as Quay.

Procedure

1. From the OpenShift AI dashboard, click **Settings → Model resources and operations→ Serving runtimes**.
   The **Serving runtimes** page opens and shows the model-serving runtimes that are already installed and enabled.
2. To add a custom runtime, choose one of the following options:
   - To start with an existing runtime (for example, **vLLM NVIDIA GPU ServingRuntime for KServe**), click the action menu (⋮) next to the existing runtime and then click **Duplicate**.
   - To add a new custom runtime, click **Add serving runtime**.
3. In the **Select the model serving platforms this runtime supports** list, select **Single-model serving platform**.
4. In the **Select the API protocol this runtime supports** list, select **REST** or **gRPC**.
5. Optional: If you started a new runtime (rather than duplicating an existing one), add your code by choosing one of the following options:
   - **Upload a YAML file**
     a. Click **Upload files**.
     b. In the file browser, select a YAML file on your computer.

The embedded YAML editor opens and shows the contents of the file that you uploaded.

- **Enter YAML code directly in the editor**
  a. Click **Start from scratch**.
  b. Enter or paste YAML code directly in the embedded editor.

NOTE

In many cases, creating a custom runtime will require adding new or custom parameters to the **env** section of the **ServingRuntime** specification.

6. Click **Add**.
   The **Serving runtimes** page opens and shows the updated list of runtimes that are installed. Observe that the custom runtime that you added is automatically enabled. The API protocol that you specified when creating the runtime is shown.
7. Optional: To edit your custom runtime, click the action menu (⋮) and select **Edit**.

Verification

- The custom model-serving runtime that you added is shown in an enabled state on the **Serving runtimes** page.

CHAPTER 2. MANAGING AND MONITORING MODELS

As a cluster administrator, you can configure monitoring, deploy models across multiple GPU nodes and set up a Grafana dashboard to visualize real-time metrics, among other tasks.

2.1. SETTING A TIMEOUT FOR KSERVE

When deploying large models or using node autoscaling with KServe, the operation may time out before a model is deployed because the default **progress-deadline** that KNative Serving sets is 10 minutes.

If a pod using KNative Serving takes longer than 10 minutes to deploy, the pod might be automatically marked as failed. This can happen if you are deploying large models that take longer than 10 minutes to pull from S3-compatible object storage or if you are using node autoscaling to reduce the consumption of GPU nodes.

To resolve this issue, you can set a custom **progress-deadline** in the KServe **InferenceService** for your application.

Prerequisites

- You have namespace edit access for your OpenShift cluster.

Procedure

1. Log in to the OpenShift console as a cluster administrator.
2. Select the project where you have deployed the model.
3. In the **Administrator** perspective, click **Home → Search**.
4. From the **Resources** dropdown menu, search for **InferenceService**.
5. Under **spec.predictor.annotations**, modify the **serving.knative.dev/progress-deadline** with the new timeout:

```
apiVersion: serving.kserve.io/v1alpha1
kind: InferenceService
metadata:
  name: my-inference-service
spec:
  predictor:
    annotations:
      serving.knative.dev/progress-deadline: 30m
```

NOTE

Ensure that you set the **progress-deadline** on the **spec.predictor.annotations** level, so that the KServe **InferenceService** can copy the **progress-deadline** back to the KNative Service object.

2.2. DEPLOYING MODELS BY USING MULTIPLE GPU NODES

Deploy models across multiple GPU nodes to handle large models, such as large language models (LLMs).

You can serve models on Red Hat OpenShift AI across multiple GPU nodes using the vLLM serving framework. Multi-node inferencing uses the **vllm-multinode-runtime** custom runtime, which uses the same image as the **vLLM NVIDIA GPU ServingRuntime for KServe** runtime and also includes information necessary for multi-GPU inferencing.

You can deploy the model from a persistent volume claim (PVC) or from an Open Container Initiative (OCI) container image.

IMPORTANT

Deploying models by using multiple GPU nodes is currently available in Red Hat OpenShift AI as a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have enabled the operators for your GPU type, such as Node Feature Discovery Operator, NVIDIA GPU Operator. For more information about enabling accelerators, see Enabling accelerators.
  - You are using an NVIDIA GPU (**nvidia.com/gpu**).
  - You have specified the GPU type through either the **ServingRuntime** or **InferenceService**. If the GPU type specified in the **ServingRuntime** differs from what is set in the **InferenceService**, both GPU types are assigned to the resource and can cause errors.
- You have enabled KServe on your cluster.
- You have only one head pod in your setup. Do not adjust the replica count using the **min_replicas** or **max_replicas** settings in the **InferenceService**. Creating additional head pods can cause them to be excluded from the Ray cluster.
- **To deploy from a PVC**: You have a persistent volume claim (PVC) set up and configured for ReadWriteMany (RWX) access mode.
- **To deploy from an OCI container image**
  - You have stored a model in an OCI container image.

- If the model is stored in a private OCI repository, you have configured an image pull secret.

Procedure

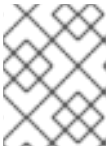
1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Select or create a namespace for deploying the model. For example, run the following command to create the **kserve-demo** namespace:

```
oc new-project kserve-demo
```

3. **(Deploying a model from a PVC only)**Create a PVC for model storage in the namespace where you want to deploy the model. Create a storage class using **Filesystem volumeMode** and use this storage class for your PVC. The storage size must be larger than the size of the model files on disk. For example:



NOTE

If you have already configured a PVC or are deploying a model from an OCI container image, you can skip this step.

```
kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: granite-8b-code-base-pvc
spec:
  accessModes:
    - ReadWriteMany
  volumeMode: Filesystem
  resources:
    requests:
      storage: <model size>
  storageClassName: <storage class>
```

a. Create a pod to download the model to the PVC you created. Update the sample YAML with your bucket name, model path, and credentials:

```
apiVersion: v1
kind: Pod
metadata:
  name: download-granite-8b-code
  labels:
    name: download-granite-8b-code
spec:
  volumes:
    - name: model-volume
      persistentVolumeClaim:
        claimName: granite-8b-code-base-pvc
  restartPolicy: Never
  initContainers:
```

```
    - name: fix-volume-permissions
      image:
quay.io/quay/busybox@sha256:92f3298bf80a1ba949140d77987f5de081f010337880cd771
f7e7fc928f8c74d
      command: ["sh"]
      args: ["-c", "mkdir -p /mnt/models/$(MODEL_PATH) && chmod -R 777 /mnt/models"]
(1)
      volumeMounts:
        - mountPath: "/mnt/models/"
          name: model-volume
      env:
        - name: MODEL_PATH
          value: <model path> (2)
  containers:
    - resources:
        requests:
          memory: 40Gi
      name: download-model
      imagePullPolicy: IfNotPresent
      image: quay.io/opendatahub/kserve-storage-initializer:v0.14 (3)
      args:
        - 's3://$(BUCKET_NAME)/$(MODEL_PATH)/'
        - /mnt/models/$(MODEL_PATH)
      env:
        - name: AWS_ACCESS_KEY_ID
          value: <id> (4)
        - name: AWS_SECRET_ACCESS_KEY
          value: <secret> (5)
        - name: BUCKET_NAME
          value: <bucket_name> (6)
        - name: MODEL_PATH
          value: <model path> (7)
        - name: S3_USE_HTTPS
          value: "1"
        - name: AWS_ENDPOINT_URL
          value: <AWS endpoint> (8)
        - name: awsAnonymousCredential
          value: 'false'
        - name: AWS_DEFAULT_REGION
          value: <region> (9)
        - name: S3_VERIFY_SSL
          value: 'true' (10)
      volumeMounts:
        - mountPath: "/mnt/models/"
          name: model-volume
```

(1) The **chmod** operation is permitted only if your pod is running as root. Remove `chmod -R 777` from the arguments if you are not running the pod as root.

(2) (7) Specify the path to the model.

(3) The value for **containers.image**, located in your **InferenceService**. To access this value, run the following command: **oc get configmap inferenceservice-config -n redhat-ods-operator -oyaml | grep kserve-storage-initializer:**

4 The access key ID to your S3 bucket.

5 The secret access key to your S3 bucket.

6 The name of your S3 bucket.

8 The endpoint to your S3 bucket.

9 The region for your S3 bucket if using an AWS S3 bucket. If using other S3-compatible storage, such as ODF or Minio, you can remove the **AWS_DEFAULT_REGION** environment variable.

10 If you encounter SSL errors, change **S3_VERIFY_SSL** to **false**.

4. Create the **vllm-multinode-runtime** custom runtime in your project namespace:

```
oc process vllm-multinode-runtime-template -n redhat-ods-applications|oc apply -n kserve-demo -f -
```

5. Deploy the model using the following **InferenceService** configuration:

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  annotations:
    serving.kserve.io/deploymentMode: RawDeployment
    serving.kserve.io/autoscalerClass: external
  name: <inference service name>
spec:
  predictor:
    model:
      modelFormat:
        name: vLLM
      runtime: vllm-multinode-runtime
      storageUri: <storage_uri_path> 1
    workerSpec: {} 2
```

1 Specify the path to your model based on your deployment method:

- For PVC: **pvc://<pvc_name>/<model_path>**
- For an OCI container image **oci://<registry_host>/<org_or_username>/<repository_name><tag_or_digest>**

2 The following configuration can be added to the **InferenceService**:

- **workerSpec.tensorParallelSize**: Determines how many GPUs are used per node. The GPU type count in both the head and worker node deployment resources is updated automatically. Ensure that the value of **workerSpec.tensorParallelSize** is at least **1**.
- **workerSpec.pipelineParallelSize**: Determines how many nodes are used to balance the model in deployment. This variable represents the total number of nodes, including both the head and worker nodes. Ensure that the value of

workerSpec.pipelineParallelSize is at least **2**. Do not modify this value in production environments.

NOTE

You may need to specify additional arguments, depending on your environment and model size.

6. Deploy the model by applying the **InferenceService** configuration:

```
oc apply -f <inference-service-file.yaml>
```

Verification

To confirm that you have set up your environment to deploy models on multiple GPU nodes, check the GPU resource status, the **InferenceService** status, the Ray cluster status, and send a request to the model.

- Check the GPU resource status:
  - Retrieve the pod names for the head and worker nodes:

```
# Get pod name
podName=$(oc get pod -l app=isvc.granite-8b-code-base-pvc-predictor --no-headers|cut
-d' ' -f1)
workerPodName=$(oc get pod -l app=isvc.granite-8b-code-base-pvc-predictor-worker --
no-headers|cut -d' ' -f1)

oc wait --for=condition=ready pod/${podName} --timeout=300s
# Check the GPU memory size for both the head and worker pods:
echo "### HEAD NODE GPU Memory Size"
kubectl exec $podName -- nvidia-smi
echo "### Worker NODE GPU Memory Size"
kubectl exec $workerPodName -- nvidia-smi
```

Sample response

```
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.90.07              Driver Version: 550.90.07      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M.
|
|                                         |                        |               MIG M. |
|=========================================+========================+==
==================|
|   0  NVIDIA A10G                    On  |   00000000:00:1E.0 Off |                    0 |
|  0%   33C    P0             71W /  300W |19031MiB /  23028MiB <1>|      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+
        ...
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.90.07              Driver Version: 550.90.07      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
```

```
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M.
|
|                                         |                      |               MIG M. |
|=========================================+======================+==
==================|
|   0  NVIDIA A10G                    On  |   00000000:00:1E.0 Off |                    0 |
|  0%   30C    P0              69W /  300W |18959MiB /  23028MiB <2>|      0%      Default |
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+
```

Confirm that the model loaded properly by checking the values of <1> and <2>. If the model did not load, the value of these fields is **0MiB**.

- Verify the status of your **InferenceService** using the following command: NOTE: In the Technology Preview, you can only use port forwarding for inferencing.

```
oc wait --for=condition=ready pod/${podName} -n $DEMO_NAMESPACE --timeout=300s
export MODEL_NAME=granite-8b-code-base-pvc
```

Sample response

```
  NAME                URL                                                READY   PREV   LATEST
PREVROLLEDOUTREVISION   LATESTREADYREVISION                    AGE
  granite-8b-code-base-pvc   http://granite-8b-code-base-pvc.default.example.com
```

- Send a request to the model to confirm that the model is available for inference:

```
oc wait --for=condition=ready pod/${podName} -n vllm-multinode --timeout=300s

oc port-forward $podName 8080:8080 &

curl http://localhost:8080/v1/completions \
      -H "Content-Type: application/json" \
      -d "{
          'model': "$MODEL_NAME",
          'prompt': 'At what temperature does Nitrogen boil?',
          'max_tokens': 100,
          'temperature': 0
       }"
```

2.3. CONFIGURING AN INFERENCE SERVICE FOR KUEUE

To queue your inference service workloads and manage their resources, add the **kueue.x-k8s.io/queue-name** label to the service's metadata. This label directs the workload to a specific **LocalQueue** for management and is required only if your project is enabled for Kueue. For more information, see Managing workloads with Kueue.

Prerequisites

- You have permissions to edit resources in the project where the model is deployed.

- As a cluster administrator, you have installed and activated the Red Hat build of Kueue Operator as described in Configuring workload management with Kueue.

Procedure

To configure the inference service, complete the following steps:

1. Log in to the OpenShift console.
2. In the Administrator perspective, navigate to your project and locate the **InferenceService** resource for your model.
3. Click the name of the InferenceService to view its details.
4. Select the **YAML** tab to open the editor.
5. In the **metadata** section, add the **kueue.x-k8s.io/queue-name** label under **labels**. Replace <local-queue-name> with the name of your target **LocalQueue**.

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: <model-name>
  namespace: <project-namespace>
  labels:
    kueue.x-k8s.io/queue-name: <local-queue-name>
...
```

6. Click **Save**.

Verification

- The workload is submitted to the **LocalQueue** specified in the **kueue.x-k8s.io/queue-name** label.
- The workload starts when the required cluster resources are available and admitted by the queue.
- Optional: To verify, run the following command and review the **Admitted Workloads** section:

```
$ oc describe localqueue <local-queue-name> -n <project-namespace>
```

2.4. CONFIGURING AN INFERENCE SERVICE FOR SPYRE

IMPORTANT

Support for IBM Spyre AI Accelerators on x86 is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

If you are deploying a model using a hardware profile that relies on Spyre schedulers, you must manually edit the **InferenceService** YAML after deployment to add the required scheduler name and tolerations. This step is necessary because the user interface does not currently provide an option to specify a custom scheduler.

Prerequisites

- You have read the Spyre Operator for IBM Z documentation.
- You have deployed a model on OpenShift by using vLLM ServingRuntime:
  - For X86 deployments, you have used the **vLLM Spyre AI Accelerator ServingRuntime for KServe** runtime.
  - For IBM Z (s390x) deployments, you have used the **vLLM Spyre s390x ServingRuntime for KServe** runtime.
- You have privileges to edit resources in the project where the model is deployed.

Procedure

To configure the inference service, complete the following steps:

1. Log in to the OpenShift console.
2. From the perspective dropdown menu, select **Administrator**.
3. From the **Project** dropdown menu, select the project where your model is deployed.
4. Navigate to **Home** > **Search**.
5. From the **Resources** dropdown menu, select **InferenceService**.
6. Click the name of the **InferenceService** resource associated with your model.
7. Select the **YAML** tab.

IMPORTANT

If you are deploying to IBM Z (s390x) note the following platform-specific requirements:

- IBM Z only supports continuous batching for this runtime. Ensure that you add the following runtime arguments in the **args** section:
  - **--max_model_len**
  - **--max-num-seqs=4**
  - **--tensor-parallel-size**.
- To use prebuilt model caches for OpenShift AI Inference Server 3.2.5 images, ensure that you add the following runtime arguments in the **args** section:
  - **--max_model_len=32768**
  - **--max-num-seqs=32**
  - **--tensor-parallel-size=4**
- The toleration key differs for the following platforms:
  - For x86 nodes, use the toleration key **ibm.com/spyre_pf**.
  - For IBM Z (s390x), only Virtual Function (VF) mode is supported for IBM Spyre devices. Therefore, use the VF toleration key for IBM Z: **ibm.com/spyre_vf**.

8. Edit the **spec.predictor** section to add the **schedulerName** and **tolerations** fields as shown in the following examples:

   - Example YAML code for the vLLM IBM Spyre AI Accelerator ServingRuntime on x86 platforms:

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
spec:
  predictor:
    schedulerName: spyre-scheduler
    tolerations:
    - effect: NoSchedule
      key: ibm.com/spyre_pf
      operator: Exists
```

   - Example YAML code for the vLLM IBM Spyre s390x ServingRuntime on IBM Z s390x platforms:

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
spec:
  predictor:
    # Continuous batching arguments required for IBM Z
    containers:
```

```
- name: kserve-container
  args:
  - "--max_model_len=2048"
  - "--max-num-seqs=4"
  - "--tensor-parallel-size=1"
  # other container args and image config as required
schedulerName: spyre-scheduler
tolerations:
- effect: NoSchedule
  key: ibm.com/spyre_vf
  operator: Exists
```

9. Click **Save**.

Verification

After you save the YAML, the existing pod for the model is terminated and a new pod is created.

1. Navigate to **Workloads** > **Pods**.
2. Click the new pod for your model to view its details.
3. On the **Details** page, verify that the pod is running on a Spyre node by checking the **Node** information.

2.5. OPTIMIZING PERFORMANCE AND TUNING

You can optimize and tune your deployed models to balance speed, efficiency, and cost for different use cases.

To evaluate a model's inference performance, consider these key metrics:

- Latency: The time it takes to generate a response, which is critical for real-time applications. This includes ▌Time-to-First-Token (TTFT) and Inter-Token Latency (ITL).
- Throughput: The overall efficiency of the model server, measured in Tokens per Second (TPS) or Requests per Second (RPS).▌
- Cost per million tokens: The cost-effectiveness of the model's inference.▌▌

Performance is influenced by factors like model size, available GPU memory, and input sequence length, especially for applications like text-summarization and retrieval-augmented generation (RAG). To meet your performance requirements, you can use techniques such as quantization to reduce memory needs or parallelism to distribute very large models across multiple GPUs.

2.5.1. Determining GPU requirements for LLM-powered applications

There are several factors to consider when choosing GPUs for applications powered by a Large Language Model (LLM) hosted on OpenShift AI.

The following guidelines help you determine the hardware requirements for your application, depending on the size and expected usage of your model.

- **Estimating memory needs**: A general rule of thumb is that a model with **N** parameters in 16-bit precision requires approximately **2N** bytes of GPU memory. For example, an 8-billion-parameter model requires around 16GB of GPU memory, while a 70-billion-parameter model requires

around 140GB.

- **Quantization**: To reduce memory requirements and potentially improve throughput, you can use quantization to load or run the model at lower-precision formats such as INT8, FP8, or INT4. This reduces the memory footprint at the expense of a slight reduction in model accuracy.

NOTE

The **vLLM ServingRuntime for KServe**model-serving runtime supports several quantization methods. For more information about supported implementations and compatible hardware, see link:https://docs.vllm.ai/en/latest/features/quantization/#supported-hardware [Supported hardware for quantization kernels].

- **Additional memory for key-value cache** In addition to model weights, GPU memory is also needed to store the attention key-value (KV) cache, which increases with the number of requests and the sequence length of each request. This can impact performance in real-time applications, especially for larger models.

- **Recommended GPU configurations**:
  - **Small Models (1B–8B parameters)**: For models in the range, a GPU with 24GB of memory is generally sufficient to support a small number of concurrent users.
  - **Medium Models (10B–34B parameters)**
    - Models under 20B parameters require at least 48GB of GPU memory.
    - Models that are between 20B - 34B parameters require at least 80GB or more of memory in a single GPU.
  - **Large Models (70B parameters)**: Models in this range may need to be distributed across multiple GPUs by using tensor parallelism techniques. Tensor parallelism allows the model to span multiple GPUs, improving inter-token latency and increasing the maximum batch size by freeing up additional memory for KV cache. Tensor parallelism works best when GPUs have fast interconnects such as an NVLink.
  - **Very Large Models (405B parameters)** For extremely large models, quantization is recommended to reduce memory demands. You can also distribute the model using pipeline parallelism across multiple GPUs, or even across two servers. This approach allows you to scale beyond the memory limitations of a single server, but requires careful management of inter-server communication for optimal performance.

For best results, start with smaller models and then scale up to larger models as required, using techniques such as parallelism and quantization to meet your performance and memory requirements.

Additional resources

- Distributed serving

2.5.2. Performance considerations for text-summarization and retrieval-augmented generation (RAG) applications

There are additional factors that need to be taken into consideration for text-summarization and RAG applications, as well as for LLM-powered services that process large documents uploaded by users.

- **Longer Input Sequences**: The input sequence length can be significantly longer than in a typical chat application, if each user query includes a large prompt or a large amount of context such as an uploaded document. The longer input sequence length increases the **prefill time**, the time the model takes to process the initial input sequence before generating a response, which can then lead to a higher **Time-to-First-Token (TTFT)**. A longer TTFT may impact the responsiveness of the application. Minimize this latency for optimal user experience.

- **KV Cache Usage**: Longer sequences require more GPU memory for the **key-value (KV) cache**. The KV cache stores intermediate attention data to improve model performance during generation. A high KV cache utilization per request requires a hardware setup with sufficient GPU memory. This is particularly crucial if multiple users are querying the model concurrently, as each request adds to the total memory load.

- **Optimal Hardware Configuration**: To maintain responsiveness and avoid memory bottlenecks, select a GPU configuration with sufficient memory. For instance, instead of running an 8B model on a single 24GB GPU, deploying it on a larger GPU (e.g., 48GB or 80GB) or across multiple GPUs can improve performance by providing more memory headroom for the KV cache and reducing inter-token latency. Multi-GPU setups with tensor parallelism can also help manage memory demands and improve efficiency for larger input sequences.

In summary, to ensure optimal responsiveness and scalability for document-based applications, you must prioritize hardware with high GPU memory capacity and also consider multi-GPU configurations to handle the increased memory requirements of long input sequences and KV caching.

2.5.3. Inference performance metrics

Latency, **throughput** and **cost per million tokens** are key metrics to consider when evaluating the response generation efficiency of a model during inferencing. These metrics provide a comprehensive view of a model's inference performance and can help balance speed, efficiency, and cost for different use cases.

2.5.3.1. Latency

Latency is critical for interactive or real-time use cases, and is measured using the following metrics:

- **Time-to-First-Token (TTFT)**: The delay in milliseconds between the initial request and the generation of the first token. This metric is important for streaming responses.

- **Inter-Token Latency (ITL)**: The time taken in milliseconds to generate each subsequent token after the first, also relevant for streaming.

- **Time-Per-Output-Token (TPOT)**: For non-streaming requests, the average time taken in milliseconds to generate each token in an output sequence.

2.5.3.2. Throughput

Throughput measures the overall efficiency of a model server and is expressed with the following metrics:

- **Tokens per Second (TPS)**: The total number of tokens generated per second across all active requests.

- **Requests per Second (RPS)**: The number of requests processed per second. RPS, like response time, is sensitive to sequence length.

2.5.3.3. Cost per million tokens

Cost per Million Tokensmeasures the cost-effectiveness of a model's inference, indicating the expense incurred per million tokens generated. This metric helps to assess both the economic feasibility and scalability of deploying the model.

2.5.4. Configuring metrics-based autoscaling

IMPORTANT

Metrics-based autoscaling is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

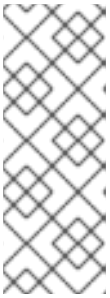
For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

Knative-based autoscaling is not available in KServe RawDeployment mode. However, you can enable metrics-based autoscaling for an inference service in this mode. Metrics-based autoscaling helps you efficiently manage accelerator resources, lower operational costs, and ensure that your inference services meet performance requirements.

To set up autoscaling for your inference service in KServe RawDeployment mode, install and configure the OpenShift Custom Metrics Autoscaler (CMA), which is based on Kubernetes Event-driven Autoscaling (KEDA). You can then use various model runtime metrics available in OpenShift Monitoring to trigger autoscaling of your inference service, such as KVCache utilization, Time to First Token (TTFT), and Concurrency.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the CMA operator on your cluster. For more information, see Installing the custom metrics autoscaler.
- You have installed the cert-manager Operator in OpenShift by using the web console as described in Installing the cert-manager Operator for Red Hat OpenShift .



NOTE

  - You must configure the **KedaController** resource after installing the CMA operator.
  - The **odh-controller** automatically creates the **TriggerAuthentication**, **ServiceAccount**, **Role**, **RoleBinding**, and **Secret** resources to allow CMA access to OpenShift Monitoring metrics.

- You have enabled User Workload Monitoring (UWM) for your cluster. For more information, see Configuring user workload monitoring.
- You have deployed a model on the model serving platform.

Procedure

1. Log in to the OpenShift console as a cluster administrator.
2. In the **Administrator** perspective, click **Home → Search**.
3. Select the project where you have deployed your model.
4. From the **Resources** dropdown menu, select **InferenceService**.
5. Click the **InferenceService** for your deployed model and then click **YAML**.
6. Under **spec.predictor**, define a metric-based autoscaling policy similar to the following example:

```
kind: InferenceService
metadata:
  name: my-inference-service
  namespace: my-namespace
  annotations:
    serving.kserve.io/autoscalerClass: keda
spec:
  predictor:
    minReplicas: 1
    maxReplicas: 5
    autoscaling:
      metrics:
        - type: External
          external:
            metric:
              backend: "prometheus"
              serverAddress: "https://thanos-querier.openshift-monitoring.svc:9092"
              query: vllm:num_requests_waiting
          authenticationRef:
            name: inference-prometheus-auth
          authModes: bearer
          target:
            type: Value
            value: 2
```

   The example configuration sets up the inference service to autoscale between 1 and 5 replicas based on the number of requests waiting to be processed, as indicated by the **vllm:num_requests_waiting** metric.

7. Click **Save**.

Verification

- Confirm that the KEDA **ScaledObject** resource is created:

```
oc get scaledobject -n <namespace>
```

2.5.5. Guidelines for metrics-based autoscaling

You can use metrics-based autoscaling to scale your AI workloads based on latency or throughput-focused Service Level Objectives (SLOs) as opposed to traditional request concurrency. Metrics-based autoscaling is based on Kubernetes Event-driven Autoscaling (KEDA).

Traditional scaling methods, which depend on factors such as request concurrency, request rate, or CPU utilization, are not effective for scaling LLM inference servers that operate on GPUs. In contrast, vLLM capacity is determined by the size of the GPU and the total number of tokens processed simultaneously. You can use custom metrics to help with autoscaling decisions to meet your SLOs.

The following guidelines can help you autoscale AI inference workloads, including selecting metrics, defining sliding windows, configuring HPA scale-down settings, and taking model size into account for optimal scaling performance.

2.5.5.1. Choosing metrics for latency and throughput-optimized scaling

For latency-sensitive applications, choose scaling metrics depending on the characteristics of the requests:

- When sequence lengths vary, use service level objectives (SLOs) for Time to First Token (TTFT) and Inter-Token Latency (ITL). These metrics provide more scaling signals because they are less affected by changes in sequence length.
- Use **end-to-end request latency** to trigger autoscaling when requests have similar sequence lengths.

End-to-end (e2e) request latency depends on sequence length, posing challenges for use cases with high variance in input/output token counts. A 10 token completion and a 2000 token completion will have vastly different latencies even under identical system conditions. To maximize throughput without latency constraints, use the **vllm:num_requests_waiting > 0.1** metric (KEDA **scaledObject** does not support a threshold of 0) to scale your workloads. This metric scales up the system as soon as a request is queued, which maximizes utilization and prevents a backlog. This strategy works best when input and output sequence lengths are consistent.

To build effective metrics-based autoscaling, follow these best practices:

- Select the right metrics:
  - Analyze your load patterns to determine sequence length variance.
  - Choose TTFT/ITL for high-variance workloads, and E2E latency for uniform workloads.
  - Implement multiple metrics with different priorities for robust scaling decisions.
- Progressively tune configurations:
  - Start with conservative thresholds and longer windows.
  - Monitor scaling behavior and SLO compliance over time.
  - Optimize the configuration based on observed patterns and business needs.
- Validate behavior through testing:
  - Run load tests with realistic sequence length distributions.
  - Validate scaling under various traffic patterns.
  - Test edge cases, such as traffic spikes and gradual load increases.

2.5.5.2. Choosing the right sliding window

The sliding window length is the time period over which metrics are aggregated or evaluated to make scaling decisions. The length of the sliding window length affects scaling responsiveness and stability.

The ideal window length depends on the metric you use:

- For Time to First Token (TTFT) and Inter-Token Latency (ITL) metrics, you can use shorter windows (1-2 minutes) because they are less noisy.
- For end-to-end latency metrics, you need longer windows (4-5 minutes) to account for variations in sequence length.

Window length	Characteristics	Best for
Short (Less than 30 seconds)	Does not effectively trigger autoscaling if the metric scraping interval is too long.	Not recommended.
Medium (60 seconds)	Responds quickly to load changes, but may lead to higher costs. Can cause rapid scaling up and down, also known as thrashing.	Workloads with sharp, unpredictable spikes.
Long (Over 4 minutes)	Balances responsiveness and stability while reducing unnecessary scaling. Might miss brief spikes and adapt slowly to load changes.	Production workloads with moderate variability.

2.5.5.3. Optimizing HPA scale-down configuration

Effective scale-down configuration is crucial for cost optimization and resource efficiency. It requires balancing the need to quickly terminate idle pods to reduce cluster load, with the consideration of maintaining them to avoid cold startup times. The Horizontal Pod Autoscaler (HPA) configuration for scale-down plays a critical role in removing idle pods promptly and preventing unnecessary resource usage.

You can control the HPA scale-down behavior by managing the KEDA **scaledObject** custom resource (CR). This Custom Resource (CR) enables event-driven autoscaling for a specific workload.

To set the time that the HPA waits before scaling down, adjust the **stabilizationWindowSeconds** field as shown in the following example:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: my-app-scaler
spec:
  advanced:
    horizontalPodAutoscalerConfig:
      behavior:
        scaleDown:
          stabilizationWindowSeconds: 300
```

2.5.5.4. Considering model size for optimal scaling

Model size affects autoscaling behavior and resource use. The following table describes the typical characteristics for different model sizes and describes a scaling strategy to select when implementing metrics-based autoscaling for AI inference workloads.

Model size	Memory footprint	Scaling strategy	Cold start time
Small (Less than 3B)	Less than 6 GiB	Use aggressive scaling with lower resource buffers.	Up to 10 minutes to download and 30 seconds to load.
Medium (3B-10B)	6-20 GiB	Use a more conservative scaling strategy.	Up to 30 minutes to download and 1 minute to load.
Large (Greater than 10B)	Greater than 20 GiB	May require model sharding or quantization.	Up to several hours to download and minutes to load.

For models with fewer than 3 billion parameters, you can reduce cold start latency with the following strategies:

- Optimize container images by embedding models directly into the image instead of downloading them at runtime. You can also use multi-stage builds to reduce the final image size and use image layer caching for faster container pulls.
- Cache models on a Persistent Volume Claim (PVC) to share storage across replicas. Configure your inference service to use the PVC to access the cached model.

Additional resources

- Distributed serving

2.6. MONITORING MODELS

2.6.1. Configuring monitoring for the model serving platform

The model serving platform includes metrics for supported runtimes of the KServe component. KServe does not generate its own metrics and relies on the underlying model-serving runtimes to provide them. The set of available metrics for a deployed model depends on its model-serving runtime.

In addition to runtime metrics for KServe, you can also configure monitoring for OpenShift Service Mesh. The OpenShift Service Mesh metrics help you to understand dependencies and traffic flow between components in the mesh.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have created OpenShift Service Mesh and Knative Serving instances and installed KServe.

- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You are familiar with creating a config map for monitoring a user-defined workflow. You will perform similar steps in this procedure.
- You are familiar with enabling monitoring for user-defined projects in OpenShift. You will perform similar steps in this procedure.
- You have assigned the **monitoring-rules-view** role to users that will monitor metrics.
- You have installed the cert-manager Operator in OpenShift by using the web console as described in Installing the cert-manager Operator for Red Hat OpenShift .

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Define a **ConfigMap** object in a YAML file called **uwm-cm-conf.yaml** with the following contents:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: user-workload-monitoring-config
  namespace: openshift-user-workload-monitoring
data:
  config.yaml: |
    prometheus:
      logLevel: debug
      retention: 15d
```

   The **user-workload-monitoring-config** object configures the components that monitor user-defined projects. Observe that the retention time is set to the recommended value of 15 days.

3. Apply the configuration to create the **user-workload-monitoring-config** object.

```
$ oc apply -f uwm-cm-conf.yaml
```

4. Define another **ConfigMap** object in a YAML file called **uwm-cm-enable.yaml** with the following contents:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
```

```
data:
  config.yaml: |
    enableUserWorkload: true
```

The **cluster-monitoring-config** object enables monitoring for user-defined projects.

5. Apply the configuration to create the **cluster-monitoring-config** object.

```
$ oc apply -f uwm-cm-enable.yaml
```

6. Create **ServiceMonitor** and **PodMonitor** objects to monitor metrics in the service mesh control plane as follows:

   a. Create an **istiod-monitor.yaml** YAML file with the following contents:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: istiod-monitor
  namespace: istio-system
spec:
  targetLabels:
  - app
  selector:
    matchLabels:
      istio: pilot
  endpoints:
  - port: http-monitoring
    interval: 30s
```

   b. Deploy the **ServiceMonitor** CR in the specified **istio-system** namespace.

```
$ oc apply -f istiod-monitor.yaml
```

   You see the following output:

```
servicemonitor.monitoring.coreos.com/istiod-monitor created
```

   c. Create an **istio-proxies-monitor.yaml** YAML file with the following contents:

```
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: istio-proxies-monitor
  namespace: istio-system
spec:
  selector:
    matchExpressions:
    - key: istio-prometheus-ignore
      operator: DoesNotExist
  podMetricsEndpoints:
  - path: /stats/prometheus
    interval: 30s
```

d. Deploy the **PodMonitor** CR in the specified **istio-system** namespace.

```
$ oc apply -f istio-proxies-monitor.yaml
```

You see the following output:

```
podmonitor.monitoring.coreos.com/istio-proxies-monitor created
```

2.7. USING GRAFANA TO MONITOR MODEL PERFORMANCE

You can deploy a Grafana metrics dashboard to monitor the performance and resource usage of your models. Metrics dashboards can help you visualize key metrics for your model-serving runtimes and hardware accelerators.

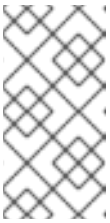
2.7.1. Deploying a Grafana metrics dashboard

You can deploy a Grafana metrics dashboard for User Workload Monitoring (UWM) to monitor performance and resource usage metrics for models deployed on the model serving platform.

You can create a Kustomize overlay, similar to this example. Use the overlay to deploy preconfigured metrics dashboards for models deployed with OpenVino Model Server (OVMS) and vLLM.

Prerequisites

- You have cluster admin privileges for your OpenShift cluster.
- A cluster admin has enabled user workload monitoring (UWM) for user-defined projects on your OpenShift cluster. For more information, see Enabling monitoring for user-defined projects and Configuring monitoring for the model serving platform.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have created an overlay to deploy a Grafana instance, similar to this example.
  - For vLLM deployments, see examples in Monitoring Dashboards.



NOTE

To view GPU metrics, you must enable the NVIDIA GPU monitoring dashboard as described in Enabling the GPU monitoring dashboard. The GPU monitoring dashboard provides a comprehensive view of GPU utilization, memory usage, and other metrics for your GPU nodes.

Procedure

1. In a terminal window, log in to the OpenShift CLI (**oc**) as a cluster administrator.
2. If you have not already created the overlay to install the Grafana operator and metrics dashboards, refer to the RHOAI UWM repository to create it.

3. Install the Grafana instance and metrics dashboards on your OpenShift cluster with the overlay that you created. Replace **<overlay-name>** with the name of your overlay.

```
oc apply -k overlays/<overlay-name>
```

4. Retrieve the URL of the Grafana instance. Replace **<namespace>** with the namespace that contains the Grafana instance.

```
oc get route -n <namespace> grafana-route -o jsonpath='{.spec.host}'
```

5. Use the URL to access the Grafana instance:

```
grafana-<namespace>.apps.example-openshift.com
```

Verification

- You can access the preconfigured dashboards available for KServe, vLLM and OVMS on the Grafana instance.

2.7.2. Deploying a vLLM/GPU metrics dashboard on a Grafana instance

Deploy Grafana boards to monitor accelerator and vLLM performance metrics.

Prerequisites

- You have deployed a Grafana metrics dashboard, as described in Deploying a Grafana metrics dashboard.
- You can access a Grafana instance.
- You have installed **envsubst**, a command-line tool used to substitute environment variables in configuration files. For more information, see the GNU **gettext** documentation.

Procedure

1. Define a **GrafanaDashboard** object in a YAML file, similar to the following examples:
   a. To monitor NVIDIA accelerator metrics, see **nvidia-vllm-dashboard.yaml**.
   b. To monitor AMD accelerator metrics, see **amd-vllm-dashboard.yaml**.
   c. To monitor Intel accelerator metrics, see **gaudi-vllm-dashboard.yaml**.
   d. To monitor vLLM metrics, see **grafana-vllm-dashboard.yaml**.
2. Create an **inputs.env** file similar to the following example. Replace the **NAMESPACE** and **MODEL_NAME** parameters with your own values:

```
NAMESPACE=<namespace> 1
MODEL_NAME=<model-name> 2
```

1 **NAMESPACE** is the target namespace where the model will be deployed.

2 **MODEL_NAME** is the model name as defined in your InferenceService. The model name is also used to filter the pod name in the Grafana dashboard.

3. Replace the **NAMESPACE** and **MODEL_NAME** parameters in your YAML file with the values from the **inputs.env** file by performing the following actions:

   a. Export the parameters described in the **inputs.env** as environment variables:

   ```
   export $(cat inputs.env | xargs)
   ```

   b. Update the following YAML file, replacing the **${NAMESPACE}** and **${MODEL_NAME}** variables with the values of the exported environment variables, and **dashboard_template.yaml** with the name of the **GrafanaDashboard** object YAML file that you created earlier:

   ```
   envsubst '${NAMESPACE} ${MODEL_NAME}' < dashboard_template.yaml > dashboard_template-replaced.yaml
   ```

4. Confirm that your YAML file contains updated values.

5. Deploy the dashboard object:

   ```
   oc create -f dashboard_template-replaced.yaml
   ```

Verification

You can see the accelerator and vLLM metrics dashboard on your Grafana instance.

2.7.3. Grafana metrics

You can use Grafana boards to monitor the accelerator and vLLM performance metrics. The **datasource**, **instance** and **gpu** are variables defined inside the board.

2.7.3.1. Accelerator metrics

Track metrics on your accelerators to ensure the health of the hardware.

NVIDIA GPU utilization

Tracks the percentage of time the GPU is actively processing tasks, indicating GPU workload levels.

Query

```
DCGM_FI_DEV_GPU_UTIL{instance=~"$instance", gpu=~"$gpu"}
```

NVIDIA GPU memory utilization

Compares memory usage against free memory, which is critical for identifying memory bottlenecks in GPU-heavy workloads.

Query

```
DCGM_FI_DEV_POWER_USAGE{instance=~"$instance", gpu=~"$gpu"}
```

-

Sum

```
sum(DCGM_FI_DEV_POWER_USAGE{instance=~"$instance", gpu=~"$gpu"})
```

NVIDIA GPU temperature

Ensures the GPU operates within safe thermal limits to prevent hardware degradation.

Query

```
DCGM_FI_DEV_GPU_TEMP{instance=~"$instance", gpu=~"$gpu"}
```

Avg

```
avg(DCGM_FI_DEV_GPU_TEMP{instance=~"$instance", gpu=~"$gpu"})
```

NVIDIA GPU throttling

GPU throttling occurs when the GPU automatically reduces the clock to avoid damage from overheating.

You can access the following metrics to identify GPU throttling:

- **GPU temperature**: Monitor the GPU temperature. Throttling often occurs when the GPU reaches a certain temperature, for example, 85-90°C.
- **SM clock speed**: Monitor the core clock speed. A significant drop in the clock speed while the GPU is under load indicates throttling.

2.7.3.2. CPU metrics

You can track metrics on your CPU to ensure the health of the hardware.

CPU utilization

Tracks CPU usage to identify workloads that are CPU-bound.

Query

```
sum(rate(container_cpu_usage_seconds_total{namespace="$namespace", pod=~"$model_name.*"}
[5m])) by (namespace)
```

CPU-GPU bottlenecks

A combination of CPU throttling and GPU usage metrics to identify resource allocation inefficiencies. The following table outlines the combination of CPU throttling and GPU utilizations, and what these metrics mean for your environment:

CPU throttling	GPU utilization	Meaning

CPU throttling	GPU utilization	Meaning
Low	High	System well-balanced. GPU is fully used without CPU constraints.
High	Low	CPU resources are constrained. The CPU is unable to keep up with the GPU's processing demands, and the GPU may be underused.
High	High	Workload is increasing for both CPU and GPU, and you might need to scale up resources.

Query

```
sum(rate(container_cpu_cfs_throttled_seconds_total{namespace="$namespace",
pod=~"$model_name.*"}[5m])) by (namespace)
avg_over_time(DCGM_FI_DEV_GPU_UTIL{instance=~"$instance", gpu=~"$gpu"}[5m])
```

2.7.3.3. vLLM metrics

You can track metrics related to your vLLM model.

GPU and CPU cache utilization

Tracks the percentage of GPU memory used by the vLLM model, providing insights into memory efficiency.

Query

```
sum_over_time(vllm:gpu_cache_usage_perc{namespace="${namespace}",pod=~"$model_name.*"}
[24h])
```

Running requests

The number of requests actively being processed. Helps monitor workload concurrency.

```
num_requests_running{namespace="$namespace", pod=~"$model_name.*"}
```

Waiting requests

Tracks requests in the queue, indicating system saturation.

```
num_requests_waiting{namespace="$namespace", pod=~"$model_name.*"}
```

Prefix cache hit rates

High hit rates imply efficient reuse of cached computations, optimizing resource usage.

Queries

```
vllm:gpu_cache_usage_perc{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"}
vllm:cpu_cache_usage_perc{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"}
```

Request total count

Query

```
vllm:request_success_total{finished_reason="length",namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}
```

The request ended because it reached the maximum token limit set for the model inference.

Query

```
vllm:request_success_total{finished_reason="stop",namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}
```

The request completed naturally based on the model's output or a stop condition, for example, the end of a sentence or token completion.

End-to-end latency

Measures the overall time to process a request for an optimal user experience.

Histogram queries

```
histogram_quantile(0.99, sum by(le)
(rate(vllm:e2e_request_latency_seconds_bucket{namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}[5m])))
histogram_quantile(0.95, sum by(le)
(rate(vllm:e2e_request_latency_seconds_bucket{namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}[5m])))
histogram_quantile(0.9, sum by(le)
(rate(vllm:e2e_request_latency_seconds_bucket{namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}[5m])))
histogram_quantile(0.5, sum by(le)
(rate(vllm:e2e_request_latency_seconds_bucket{namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}[5m])))
rate(vllm:e2e_request_latency_seconds_sum{namespace="$namespace",
pod=~"$model_name.*",model_name="$model_name"}[5m])
rate(vllm:e2e_request_latency_seconds_count{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"}[5m])
```

Time to first token (TTFT) latency

The time taken to generate the first token in a response.

Histogram queries

```
histogram_quantile(0.99, sum by(le)
(rate(vllm:time_to_first_token_seconds_bucket{namespace="$namespace", pod=~"$model_name.*",
```

```
model_name="$model_name"}[5m]))
histogram_quantile(0.95, sum by(le)
(rate(vllm:time_to_first_token_seconds_bucket{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"}[5m]))
histogram_quantile(0.9, sum by(le)
(rate(vllm:time_to_first_token_seconds_bucket{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"}[5m]))
histogram_quantile(0.5, sum by(le)
(rate(vllm:time_to_first_token_seconds_bucket{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"}[5m]))
rate(vllm:time_to_first_token_seconds_sum{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"}[5m])
rate(vllm:time_to_first_token_seconds_count{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"}[5m])
```

Time per output token (TPOT) latency

The average time taken to generate each output token.

Histogram queries

```
histogram_quantile(0.99, sum by(le)
(rate(vllm:time_per_output_token_seconds_bucket{namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}[5m]))
histogram_quantile(0.95, sum by(le)
(rate(vllm:time_per_output_token_seconds_bucket{namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}[5m]))
histogram_quantile(0.9, sum by(le)
(rate(vllm:time_per_output_token_seconds_bucket{namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}[5m]))
histogram_quantile(0.5, sum by(le)
(rate(vllm:time_per_output_token_seconds_bucket{namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}[5m]))
rate(vllm:time_per_output_token_seconds_sum{namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}[5m])
rate(vllm:time_per_output_token_seconds_count{namespace="$namespace",
pod=~"$model_name.*", model_name="$model_name"}[5m])
```

Prompt token throughput and generation throughput

Tracks the speed of processing prompt tokens for LLM optimization.

Queries

```
rate(vllm:prompt_tokens_total{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"}[5m])
rate(vllm:generation_tokens_total{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"}[5m])
```

Total tokens generated

Measures the efficiency of generating response tokens, critical for real-time applications.

Query

```
sum(vllm:generation_tokens_total{namespace="$namespace", pod=~"$model_name.*",
model_name="$model_name"})
```

CHAPTER 3. MANAGING AND MONITORING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM

As a cluster administrator, you can manage and monitor models on the NVIDIA NIM model serving platform. You can customize your NVIDIA NIM model selection options and enable metrics for a NIM model, among other tasks.

3.1. CUSTOMIZING MODEL SELECTION OPTIONS FOR THE NVIDIA NIM MODEL SERVING PLATFORM

The NVIDIA NIM model serving platform provides access to all available NVIDIA NIM models from the NVIDIA GPU Cloud (NGC). You can deploy a NIM model by selecting it from the **NVIDIA NIM** list in the **Deploy model** dialog. To customize the models that appear in the list, you can create a **ConfigMap** object specifying your preferred models.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have an NVIDIA Cloud Account (NCA) and can access the NVIDIA GPU Cloud (NGC) portal.
- You know the IDs of the NVIDIA NIM models that you want to make available for selection on the NVIDIA NIM model serving platform.

NOTE

  - You can find the model ID from the NGC Catalog. The ID is usually part of the URL path.
  - You can also find the model ID by using the NGC CLI. For more information, see NGC CLI reference.

- You know the name and namespace of your **Account** custom resource (CR).

Procedure

1. In a terminal window, log in to the OpenShift CLI as a cluster administrator as shown in the following example:

```
oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Define a **ConfigMap** object in a YAML file, similar to the one in the following example, containing the model IDs that you want to make available for selection on the NVIDIA NIM model serving platform:

```
apiVersion: v1
kind: ConfigMap
metadata:
 name: nvidia-nim-enabled-models
data:
 models: |-
   [
```

```
    "mistral-nemo-12b-instruct",
    "llama3-70b-instruct",
    "phind-codellama-34b-v2-instruct",
    "deepseek-r1",
    "qwen-2.5-72b-instruct"
    ]
```

3. Confirm the name and namespace of your **Account** CR:

```
oc get account -A
```

You see output similar to the following example:

```
NAMESPACE          NAME      TEMPLATE  CONFIGMAP  SECRET
redhat-ods-applications  odh-nim-account
```

4. Deploy the **ConfigMap** object in the same namespace as your **Account** CR:

```
oc apply -f <configmap-name> -n <namespace>
```

Replace *<configmap-name>* with the name of your YAML file, and *<namespace>* with the namespace of your **Account** CR.

5. Add the **ConfigMap** object that you previously created to the **spec.modelListConfig** section of your **Account** CR:

```
oc patch account <account-name> \
  --type='merge' \
   -p '{"spec": {"modelListConfig": {"name": "<configmap-name>"}}}'
```

Replace *<account-name>* with the name of your **Account** CR, and *<configmap-name>* with your **ConfigMap** object.

6. Confirm that the **ConfigMap** object is added to your **Account** CR:

```
oc get account <account-name> -o yaml
```

You see the **ConfigMap** object in the **spec.modelListConfig** section of your **Account** CR, similar to the following output:

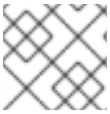
```
spec:
 enabledModelsConfig:
 modelListConfig:
  name: <configmap-name>
```

Verification

- Follow the steps to deploy a model as described in Deploying models on the NVIDIA NIM model serving platform to deploy a NIM model. You see that the **NVIDIA NIM** list in the **Deploy model** dialog displays your preferred list of models instead of all the models available in the NGC catalog.

3.2. ENABLING NVIDIA NIM METRICS FOR AN EXISTING NIM DEPLOYMENT

If you have previously deployed a NIM model in OpenShift AI, and then upgraded to 3.0, you must manually enable NIM metrics for your existing deployment by adding annotations to enable metrics collection and graph generation.



NOTE

NIM metrics and graphs are automatically enabled for new deployments in 2.17.

3.2.1. Enabling graph generation for an existing NIM deployment

The following procedure describes how to enable graph generation for an existing NIM deployment.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have an existing NIM deployment in OpenShift AI.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**).
2. Confirm the name of the **ServingRuntime** associated with your NIM deployment:

```
oc get servingruntime -n <namespace>
```

   Replace **<namespace>** with the namespace of the project where your NIM model is deployed.

3. Check for an existing **metadata.annotations** section in the **ServingRuntime** configuration:

```
oc get servingruntime -n  <namespace> <servingruntime-name> -o json | jq
'.metadata.annotations'
```

   Replace <servingruntime-name> with the name of the **ServingRuntime** from the previous step.

4. Perform one of the following actions:
   a. If the **metadata.annotations** section is not present in the configuration, add the section with the required annotations:

```
oc patch servingruntime -n <namespace> <servingruntime-name> --type json --patch \
 '[{"op": "add", "path": "/metadata/annotations", "value": {"runtimes.opendatahub.io/nvidia-
nim": "true"}}]'
```

You see output similar to the following:

```
servingruntime.serving.kserve.io/nim-serving-runtime patched
```

b. If there is an existing **metadata.annotations** section, add the required annotations to the section:

```
oc patch servingruntime -n <project-namespace> <runtime-name> --type json --patch \
 '[{"op": "add", "path": "/metadata/annotations/runtimes.opendatahub.io~1nvidia-nim",
"value": "true"}]'
```

You see output similar to the following:

```
servingruntime.serving.kserve.io/nim-serving-runtime patched
```

Verification

- Confirm that the annotation has been added to the **ServingRuntime** of your existing NIM deployment.

```
oc get servingruntime -n <namespace> <servingruntime-name> -o json | jq
'.metadata.annotations'
```

The annotation that you added is displayed in the output:

```
...
"runtimes.opendatahub.io/nvidia-nim": "true"
```

NOTE

For metrics to be available for graph generation, you must also enable metrics collection for your deployment. For more information, see Enabling metrics collection for an existing NIM deployment. endif:[]

3.2.2. Enabling metrics collection for an existing NIM deployment

To enable metrics collection for your existing NIM deployment, you must manually add the Prometheus endpoint and port annotations to the **InferenceService** of your deployment.

The following procedure describes how to add the required Prometheus annotations to the **InferenceService** of your NIM deployment.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS

- You have an existing NIM deployment in OpenShift AI.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**).
2. Confirm the name of the **InferenceService** associated with your NIM deployment:

```
oc get inferenceservice -n <namespace>
```

   Replace **<namespace>** with the namespace of the project where your NIM model is deployed.

3. Check if there is an existing **spec.predictor.annotations** section in the **InferenceService** configuration:

```
oc get inferenceservice -n <namespace> <inferenceservice-name> -o json | jq
'.spec.predictor.annotations'
```

   Replace <inferenceservice-name> with the name of the **InferenceService** from the previous step.

4. Perform one of the following actions:
   a. If the **spec.predictor.annotations** section does not exist in the configuration, add the section and required annotations:

```
oc patch inferenceservice -n <namespace> <inference-name> --type json --patch \
 '[{"op": "add", "path": "/spec/predictor/annotations", "value": {"prometheus.io/path":
"/metrics", "prometheus.io/port": "8000"}}]'
```

      The annotation that you added is displayed in the output:

```
inferenceservice.serving.kserve.io/nim-serving-runtime patched
```

   b. If there is an existing **spec.predictor.annotations** section, add the Prometheus annotations to the section:

```
oc patch inferenceservice -n <namespace> <inference-service-name> --type json --
patch \
 '[{"op": "add", "path": "/spec/predictor/annotations/prometheus.io~1path", "value":
"/metrics"},
 {"op": "add", "path": "/spec/predictor/annotations/prometheus.io~1port", "value": "8000"}]'
```

      The annotations that you added is displayed in the output:

```
inferenceservice.serving.kserve.io/nim-serving-runtime patched
```

Verification

- Confirm that the annotations have been added to the **InferenceService**.

```
oc get inferenceservice -n <namespace> <inferenceservice-name> -o json | jq
'.spec.predictor.annotations'
```

You see the annotation that you added in the output:

```
{
  "prometheus.io/path": "/metrics",
  "prometheus.io/port": "8000"
}
```